

MODULE 5.1 · Concept 1 of 7

Classical Planning

State-space search, STRIPS, and forward/backward chaining — the foundations agents build on

9 min

Duration

B2 – Understand

Bloom's

L3 – Advanced

Level

CD-000001

Prerequisite

AI-AG-IN-TH-000005

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHAT

Classical AI Planning

Deciding WHAT to do before doing it — structured, complete, and sound

What Planning Means for Agents

Deciding what to do BEFORE doing it — contrast with reactive and ReAct

Three approaches to agent decision-making:

Reactive (C03)

IF percept → THEN action

None — responds immediately

No multi-step, no foresight

ReAct (C09-C15)

Think → Act → Observe → Repeat

One step ahead per iteration

No global plan, may wander

Planning (This section)

Generate full plan → Execute steps

Complete plan before first action

Slower start, but more directed

Planning adds foresight: the agent knows the entire path before taking the first step. This prevents wasted actions.

State-Space Search

The world as states, actions as transitions — the foundation of classical planning



Key concepts:

- State: a complete description of the world at one moment (e.g., 'Block A is on Block B, Block C is on table')
- Action: a transition that changes one state to another (e.g., 'pick up Block A')
- Initial state S_0 : where the agent starts
- Goal state G : where the agent needs to arrive
- Plan: a sequence of actions $[a_1, a_2, \dots, a_n]$ that transforms S_0 into G

STRIPS Representation

Preconditions, effects, and goal states — the formal language of classical planning

STRIPS Action Schema (Stanford Research Institute Problem Solver, 1971)

Action: `pick_up(block, from_surface)`

Preconditions:

- `block` is ON `from_surface`
- `block` has NOTHING on top of it (clear)
- robot hand is EMPTY

Effects (Add):

- robot is HOLDING `block`
- `from_surface` is CLEAR

Effects (Delete):

- `block` is ON `from_surface`
- robot hand is EMPTY

Every action has: (1) conditions that must be true BEFORE it can execute, (2) things that become true AFTER, (3) things that stop being true.

Forward vs. Backward Chaining

Two search strategies for finding a plan

Forward Chaining (Progression)

- Start from initial state S_0 .
- At each step, find all applicable actions
- (whose preconditions are met).
- Apply an action $\rightarrow$ get new state S_1 .
- Repeat until goal G is reached.

✓ Intuitive — follows natural time order
✗ Large branching factor — many actions applicable at each state

Backward Chaining (Regression)

- Start from goal state G .
- Find actions whose effects achieve part of the goal.
- Add their preconditions as new sub-goals.
- Repeat until sub-goals match S_0 .

✓ More focused — only relevant actions
✗ Less intuitive — works backwards
✗ Harder to implement correctly

Modern planners often combine both: backward to identify relevant actions, forward to verify executability.

Traced Example: Blocks World

The classic planning domain — move blocks to achieve a target configuration

Goal: Stack blocks so that A is on B, B is on C, C is on table

Step	Action	Result State
Initial: C on A, A on table, B on table	—	C-on-A, A-on-table, B-on-table
Pick up C from A	<code>pick_up(C, A)</code>	Holding C, A clear
Put C on table	<code>put_down(C, table)</code>	C-on-table, A clear, B clear
Pick up B	<code>pick_up(B, table)</code>	Holding B
Stack B on C	<code>stack(B, C)</code>	B-on-C, C-on-table
Pick up A	<code>pick_up(A, table)</code>	Holding A
Stack A on B → GOAL	<code>stack(A, B)</code>	A-on-B, B-on-C, C-on-table ✓

Traced Example: Travel Planning

Multi-city trip with constraints — classical approach

Goal: Travel BLR → Mumbai → Goa → BLR within ₹15,000 and 3 days

State variables: location, budget_remaining, days_remaining, bookings[]

S_0 : location=BLR, budget=₹15000, days=3, bookings=[]

Action 1: fly(BLR→MUM, ₹4200, day1)

Pre: location=BLR, budget≥4200, days≥1

Effect: location=MUM, budget=₹10800, days=2

Action 2: train(MUM→GOA, ₹800, day2)

Pre: location=MUM, budget≥800, days≥1

Effect: location=GOA, budget=₹10000, days=1

Action 3: fly(GOA→BLR, ₹3500, day3)

Pre: location=GOA, budget≥3500, days≥1

Effect: location=BLR, budget=₹6500, days=0 → GOAL ✓

SECTION 2 — WHY IT FAILS

Limitations of Classical Planning

Complete and sound — but struggles with the real world

Completeness and Soundness – What Classical Planning Guarantees

If a plan exists, the planner WILL find it. If it finds a plan, it WILL work.

Property	Meaning	Value for Agents
Completeness	If a valid plan exists, the algorithm will find it	Guaranteed solution — no missed options
Soundness	Every plan found is actually executable	No infeasible plans — preconditions verified
Optimality	Can find the shortest/cheapest plan	Cost-efficient execution (if configured)
Deterministic	Same input always produces same plan	Predictable, reproducible behavior

The catch: These guarantees only hold when the world is fully described in formal logic. Real-world tasks with natural language, uncertain outcomes, and open-ended goals break these assumptions.

The Open-World Problem + Natural Language

Two fundamental mismatches with real agent tasks

Closed-World Assumption

Classical planning assumes everything NOT stated is FALSE. The world is fully known.

Reality: Agents operate in open worlds.

- Unknown flight delays
- Prices change while planning
- New constraints appear mid-execution
- Users change their minds

Result: Plans become invalid quickly.

Natural Language Gap

Classical planners need FORMAL input:
`pick_up(block_A, table)`

Users give NATURAL LANGUAGE:

- "Make the code more efficient"
- "Plan something fun for the weekend"
- "Fix the bug but don't break anything"

Classical planners cannot parse these.
This is where LLMs come in. → Next concept.

Where Classical Planning Still Works Today

Not obsolete — thriving in structured, well-defined domains

Domain	Why Classical Works	Example Systems
Robotics	Physical actions have clear preconditions/effects	ROS MoveIt, PDDL planners
Game AI	Game rules define a closed world perfectly	Starcraft bots, chess engines
Logistics	Routes, capacity, schedules are fully specified	UPS ORION, Amazon warehouse
Satellite ops	Orbital mechanics are deterministic	NASA EUROPA planner
Manufacturing	Assembly steps are precise and ordered	Factory automation systems

Pattern: Classical planning excels when the world is fully described, actions are deterministic, and goals are formal.

The Bridge to LLM Planning

Classical gives structure. LLMs give language understanding. Together: powerful.

Capability	Classical Planning	LLM Planning	Combined (Hybrid)
Language understanding	✗ Cannot parse NL	✓ Native NL parsing	✓ LLM parses, classical structures
Formal correctness	✓ Sound + complete	✗ May hallucinate plans	✓ LLM generates, verifier checks
Open-world handling	✗ Closed world only	✓ Handles uncertainty	✓ LLM adapts, classical validates
Scalability	✗ Exponential state space	✓ Scales with prompt size	✓ LLM prunes, classical searches

Next two concepts: LLM-Based Planning (C17) and Hybrid Planning (C18) build on this foundation.

Key Takeaways

The essential points from this concept

- Classical planning = state-space search** with STRIPS actions (preconditions + effects).
- Completeness + soundness guaranteed** — if a plan exists, the planner finds it and it works.
- Two strategies: forward chaining (from start)** and backward chaining (from goal).
- Fails on open worlds and natural language** — real tasks break the closed-world assumption.
- Still thriving in robotics, games, logistics** — anywhere the world is fully specified.

Next: AI-AG-IN-TH-000006 — LLM-Based Planning

AI-AG-IN-TH-000005 Complete

Classical Planning

Concept 1 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi